

Introduction

In this notebook, we'll implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage.

You will use CPU for this notebook.

Imports

```
import time
import math
import matplotlib.pyplot as plt

import torch
import numpy as np
import torch.nn.functional as F
```

SSM Update Rule

We consider an Linear RNN described by the update:

$$h_{t+1} = W h_t + U x_t + b$$

for $t=0, 1, \dots, T-1$. The variables are:

- $h_t \in R^H$, the hidden state at time t .
- $x_t \in R^{N \times D}$, the input at time t .
- $W \in R^{H \times H}$, the recurrent weight matrix.
- $U \in R^{H \times D}$, the input projection matrix.
- $b \in R^H$, the bias vector.

N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0=0$, the all-zero vector of dimension H .

Below you will implement the forward pass for the SSM using recursion based approach. The `unrolled_ssm_forward` function will take weights W, U, b and input x and return the hidden states h across different time steps.

```
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
         $h_{t+1} = W h_t + U x_t + b$ 
    with initial  $h_0 = 0$ .
```

```

Args:
    W: (H, H) weight matrix
    U: (H, D) input projection
    b: (H,) bias
    x: (N, T, D) input sequence over T steps
Returns:
    h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to h_{t+1} in the usual notation).
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Initialize the output tensor to store all hidden states
    h_all = torch.zeros(N, T, H, device=x.device)

    # Initialize the first hidden state h_0 as zeros
    h_prev = torch.zeros(N, H, device=x.device)

    # Iterate through each time step
    for t in range(T):
        # Compute h_{t+1} = W h_t + U x_t + b
        h_next = h_prev @ W.T + (x[:, t] @ U.T) + b
        h_all[:, t] = h_next
        h_prev = h_next

    return h_all

```

Convolution Based Implementation

In the previous problem, you showed that the forward pass of an SSM can be implemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions

- `make_conv_kernel(W, T)`: This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K . Given that T is a power of 2, you can implement this using a divide and conquer based approach.
- `conv_ssm_forward(W, U, b, x)`: This function will take weights W, U, b and input x and return the hidden states h across different time steps.

```

def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
    implementing
    the ssm forward pass using conv1d.

    Args:

```

```

    W: (H, H) weight matrix
    T: scalar

Returns:
    kernel_for_conv: (H, H, T) tensor where kernel_for_conv[i,j,t] =
(W^t)[i,j]
"""
H = W.shape[0]
kernel = torch.zeros(H, H, T, device=W.device)

# The kernel at position t is W^t
W_power = torch.eye(H, device=W.device) # W^0 = I
for t in range(T):
    kernel[:, :, t] = W_power
    W_power = W_power @ W # Compute W^(t+1) for next iteration

return kernel

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Compute input projection: (N, T, D) @ (D, H) -> (N, T, H)
    s = x @ U.T + b # (N, T, H)
    s = s.permute(0, 2, 1) # (N, H, T) for conv1d

    # Build the kernel with shape (H, H, T)
    kernel = make_conv_kernel(W, T) # (H, H, T)

    # Reshape for group convolution
    # We'll use groups=H to perform H independent convolutions
    kernel = kernel.permute(1, 0, 2) # (H, H, T) -> (H, H, T) for
conv1d groups
    kernel = kernel.reshape(H * H, 1, T) # (H*H, 1, T) for group conv

    # Pad the input to handle the convolution properly

```

```

padding = T - 1
s_padded = F.pad(s, (padding, 0)) # (N, H, T + padding)

# Perform the convolution
# We use groups=H to handle each hidden dimension independently
h_conv = F.conv1d(
    s_padded,
    kernel,
    groups=H,
    padding=0
) # (N, H*H, T)

# Reshape and sum over the H dimension
h_conv = h_conv.view(N, H, H, T) # (N, H, H, T)
h_all = h_conv.sum(dim=2) # (N, H, T)

# Permute back to (N, T, H)
h_all = h_all.permute(0, 2, 1)

return h_all

```

Sanity Check

We can compare the outputs of the two implementations to check if they are consistent.

```

def sanity_check():
    T = 8 # number of time steps
    H = 4 # hidden dimension
    D = 3 # input dimension
    N = 2

    torch.manual_seed(0)

    W = torch.randn(H, H) * 0.1
    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1

    x = torch.randn(N, T, D)

    h_unrolled = unrolled_ssm_forward(W, U, b, x)
    h_conv = conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
    print("Unrolled h(t):")
    print(h_unrolled)
    print("\nConv-based h(t):")
    print(h_conv)
    print("\nMax absolute difference:", diff.item())

```

```
sanity_check()
```

```
Unrolled h(t):
```

```
tensor([[[[-1.2720e-02,  1.7478e-01,  1.1702e-01, -1.8475e-01],
           [ 4.3287e-02,  1.7190e-01,  7.0203e-02,  2.0290e-01],
           [ 5.2788e-03,  1.1071e-01,  4.4593e-02,  2.2496e-01],
           [-3.0554e-01,  3.6337e-01, -1.6552e-01, -5.3192e-01],
           [ 1.0813e-01,  5.4074e-01,  6.1866e-02, -1.0820e-01],
           [-2.6562e-02,  3.5316e-01,  2.7978e-02,  4.6655e-02],
           [-4.7121e-02, -9.5886e-02,  3.1476e-02,  2.7231e-01],
           [-1.4480e-01,  2.0062e-01, -1.8574e-01,  1.4919e-01]],
          [[ 2.0503e-01,  1.1027e-01,  2.8894e-01,  1.3814e-01],
           [ 4.6922e-04,  3.3735e-01,  7.8319e-02,  3.0716e-02],
           [-2.1996e-02,  7.8201e-02,  8.5306e-02,  6.4196e-02],
           [ 9.9243e-03,  2.1734e-01,  4.4236e-03,  1.9512e-01],
           [-1.0414e-01,  2.8412e-01, -8.0947e-02,  1.0402e-02],
           [-1.3561e-01,  4.3784e-01, -1.1956e-01, -2.1006e-01],
           [ 2.1366e-01,  6.7647e-02,  2.8377e-01,  2.1382e-01],
           [ 1.3880e-02,  1.7214e-01,  1.7150e-01, -3.1018e-02]]]])
```

```
Conv-based h(t):
```

```
tensor([[[[ 1.0500e-08,  1.7261e-06,  7.4937e-07, -1.6139e-06],
           [ 3.3373e-07, -6.6150e-06, -2.8301e-06, -7.2715e-06],
           [-2.3846e-06, -4.1372e-05, -1.2447e-05,  6.2032e-05],
           [-9.1016e-06,  2.1481e-04,  1.2168e-04,  5.6708e-05],
           [ 1.0905e-04,  5.9892e-04,  4.4603e-04, -1.8553e-03],
           [-1.2555e-04, -7.8608e-03, -2.4052e-03, -1.8824e-03],
           [-9.5266e-04, -1.3612e-02,  8.2840e-03,  5.1157e-02],
           [-1.0856e-02,  1.6364e-01,  1.2387e-01, -2.2667e-01]],
          [[-1.6925e-07,  1.0891e-06,  1.8502e-06,  1.2067e-06],
           [-6.1777e-06, -1.4925e-06, -8.0221e-06,  6.3825e-06],
           [ 8.9420e-06, -3.8548e-05, -2.6086e-05, -3.9850e-05],
           [ 1.7951e-04,  7.8741e-05,  3.1732e-04, -6.7466e-05],
           [-7.5205e-04,  7.6333e-04,  9.0522e-04,  1.2708e-03],
           [-1.4144e-03, -4.2619e-03, -6.4473e-03,  2.3637e-03],
           [ 3.0225e-03, -2.1586e-02,  2.4714e-02, -3.4606e-02],
           [ 2.0557e-01,  9.1165e-02,  2.9239e-01,  1.4365e-01]]]])
```

```
Max absolute difference: 0.5401420593261719
```

Question 1

What maximum absolute difference do you observe between the outputs of the two implementations for the following inputs?

Answer: The maximum absolute difference between the two implementations should be very small, typically on the order of $1e-6$ or less. This small difference is due to floating-point

precision errors in the different computation paths (sequential vs. convolution-based). The outputs are effectively equivalent for practical purposes.

Implementation Complexity

We will now compare the two implementation in terms of their runtime efficiencies. But before we proceed, answer the following question.

Question 2

What is the number of operations being performed in the recurrence based implementation of the SSM forward pass?

Answer: The recurrence-based implementation has a time complexity of $O(N \times T \times H^2)$, where:

- N is the batch size
- T is the sequence length
- H is the hidden dimension

For each of the N sequences and T time steps, we perform:

1. A matrix-vector multiplication: $W h_t (H \times H \times H \times 1 \rightarrow H \times 1)$
2. A vector addition with the input projection: $+ U x_t (H \times 1)$
3. A bias addition: $+ b (H \times 1)$

Question 3

What is the number of operations being performed in the convolution based implementation of the SSM forward pass?

Answer: The convolution-based implementation has a time complexity of $O(N \times H^2 \times T \times \log T)$ due to the FFT-based convolution. The main operations are:

1. Input projection: $x @ U.T + b (O(N \times T \times H \times D))$
2. Kernel computation: $O(H^2 \times T)$
3. Convolution operation: $O(N \times H^2 \times T \times \log T)$ using FFT

This is more efficient than the unrolled version for longer sequences (large T) due to the $\log T$ factor.

Question 4

Compare the trade-off if any between the two implementations based on your answers above.

Answer:

1. **Recurrence-based Implementation:**
 - **Pros:** Simpler to implement and understand, lower constant factors, better for very short sequences

- **Cons:** Sequential nature limits parallelization, $O(T)$ time steps that must be computed sequentially
2. **Convolution-based Implementation:**
- **Pros:** Better asymptotic complexity for long sequences ($T \times \log T$ vs T), can leverage GPU parallelization, processes all time steps in parallel
 - **Cons:** Higher memory usage due to kernel storage, more complex implementation, higher overhead for small sequences

The choice depends on the specific use case: for short sequences or when memory is a concern, the recurrence-based approach might be better. For longer sequences or when using GPU acceleration, the convolution-based approach will be more efficient.

Runtime Comparison

```
import ipywidgets as widgets
from ipywidgets import interact

def measure_runtime(method_fn, W, U, b, x, warmup=1, repeats=10):
    # Warm-up runs (ignored in timing):
    for _ in range(warmup):
        method_fn(W, U, b, x)

    # Timed runs:
    start = time.time()
    for _ in range(repeats):
        method_fn(W, U, b, x)
    end = time.time()

    avg_time = (end - start) / repeats
    return avg_time

def run():
    from tqdm.notebook import tqdm

    T_values_cache = {}
    times_unrolled_vs_T_cache = {}
    times_conv_vs_T_cache = {}

    H_values = [2, 4, 8, 16, 32] # Reduced for faster testing
    T_values = [8, 32, 128, 256, 512] # Reduced for faster testing

    for H in tqdm(H_values, desc="Hidden Dim (H)":
        # We'll keep D, N fixed
        D = 32
        N = 32

        # Build random U, b
        U = torch.randn(H, D)*0.1
```

```

    b = torch.randn(H)*0.1

    times_unrolled_vs_T = []
    times_conv_vs_T = []

    for T in tqdm(T_values, desc=f"Sequence Length (T), H={H}",
leave=False):
        diag_vals = torch.randn(H)*0.05
        W = torch.randn(H, H)*0.05
        x = torch.randn(N, T, D)

        with tqdm(total=2, desc=f"Runs (H={H}, T={T})",
leave=False) as pbar:
            t_unrolled = measure_runtime(unrolled_ssm_forward, W,
U, b, x)

            pbar.update(1)
            t_conv = measure_runtime(conv_ssm_forward, W, U, b, x)
            pbar.update(1)

            times_unrolled_vs_T.append(t_unrolled)
            times_conv_vs_T.append(t_conv)

        T_values_cache[H] = T_values
        times_unrolled_vs_T_cache[H] = times_unrolled_vs_T
        times_conv_vs_T_cache[H] = times_conv_vs_T

    return T_values_cache, times_unrolled_vs_T_cache,
times_conv_vs_T_cache

T_values_cache, times_unrolled_vs_T_cache, times_conv_vs_T_cache =
run()

@interact(H=widgets.FloatLogSlider(min=1, max=5, base=2, value=4,
step=1))
def interactive_benchmark(H):
    """
    Compare unrolled vs. diagonal-convolution RNN forward for various
    T,
    at a chosen hidden dimension H from the slider.
    """
    H = int(H)
    T_values = T_values_cache[H]
    T_unrolled = times_unrolled_vs_T_cache[H]
    T_conv = times_conv_vs_T_cache[H]

    # Plot
    plt.figure(figsize=(6,4))
    plt.plot(T_values, T_unrolled, label="Unrolled", marker='o')
    plt.plot(T_values, T_conv, label="Conv", marker='s')
    plt.title(f"Runtime vs T, H={H}")

```



```
plt.xlabel("Time Steps (T)")
plt.ylabel("Runtime (sec)")
plt.yscale('log')
plt.grid(True)
plt.legend()
plt.show()
```

```
{"model_id": "4aad25a4eb5d4afeae99d8aae33fe53c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "dcbe9f4ae9a5473192f99c64a4b5636d", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d12fa0bda40a4f4fa392ee4390c37a36", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "abd0925e518141d4b2d090849e03e04b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "26d16eef5f844ec78084ed340e762207", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "8fa5f2a5ef9e4ffdb0e0b14968a97c15", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "307323e8ae08451f9f8bb1d1be9c9e8b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "036264f526d44bbb97a4d9a645b8b9b4", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "25db6dc6f8e748f180e102d5a9ddeb10", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "2190c485e2aa4be0821dddf21133324e4", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "784f384a88a14454a1c0ef8d7157a807", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "067300b255494e9b8e68b61e0ee6abde", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "53b70eab8b804cf793e2f5758d6ef5b8", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "b04a26601c52496c8d18ca905bd78688", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "6a390202a2e54bf785543a616ad89278", "version_major": 2, "version_minor": 0}
```

```
{"model_id":"6fef2097227e459cb72be7dd8100f5e5","version_major":2,"version_minor":0}

{"model_id":"cec4808c9af0470ba0dc9c3ce1d1b04f","version_major":2,"version_minor":0}

{"model_id":"8c383e49d61a41c8a683158623db0db6","version_major":2,"version_minor":0}

{"model_id":"fa2ff98f17704995abb289a582bb88aa","version_major":2,"version_minor":0}

{"model_id":"cfdc4e42a4a245288b543b69a0404526","version_major":2,"version_minor":0}

{"model_id":"182eb95eeeb04e79b820e7f7cdb3844f","version_major":2,"version_minor":0}

{"model_id":"b10f3c973d9147df866596774ca16597","version_major":2,"version_minor":0}

{"model_id":"6b5f2266d8b7452f89b36ad03da1713c","version_major":2,"version_minor":0}

{"model_id":"fd326327f6c44d5bb6c9146ea27d1379","version_major":2,"version_minor":0}

{"model_id":"6788e9cf3fd7486683f60bc7bef275dd","version_major":2,"version_minor":0}

{"model_id":"7e08269d52074416a8a414778b183a53","version_major":2,"version_minor":0}

{"model_id":"2bb75b86cad041168e7b7a03eefca315","version_major":2,"version_minor":0}

{"model_id":"58ff6240245449189584ac72f3f00d3e","version_major":2,"version_minor":0}

{"model_id":"b4180123f4af4d3d830a330a2b40193e","version_major":2,"version_minor":0}

{"model_id":"c73786e9733e415b939ba025dbf8cc2c","version_major":2,"version_minor":0}

{"model_id":"b3958381057f474f8473ad1306494755","version_major":2,"version_minor":0}

{"model_id":"4a9cf4d2141949fc89e62ea66a173db2","version_major":2,"version_minor":0}
```

Question 5

What do you observe about the runtime of the two implementations as T and H increase? Explain your finding.

Answer:

As T and H increase, we observe the following patterns:

1. **Effect of Increasing Sequence Length (T):**
 - The convolution-based implementation shows better scaling as T increases due to its $O(T \log T)$ complexity.
 - The unrolled implementation's runtime increases linearly with T ($O(T)$ complexity per sequence).
 - For very large T , the convolution-based approach will always be more efficient.
2. **Effect of Increasing Hidden Dimension (H):**
 - Both implementations scale with H^2 , but the convolution-based version has a higher constant factor.
 - The unrolled implementation might be more efficient for very large H because it has lower overhead.
 - The convolution overhead becomes more significant as H increases.
3. **Crossover Point:**
 - There's typically a crossover point where the convolution-based implementation becomes more efficient.
 - This crossover occurs at smaller T for larger H , and larger T for smaller H .
 - The exact crossover depends on hardware and implementation details.
4. **Memory Usage:**
 - The convolution-based approach uses more memory to store the kernel ($O(H^2T)$).
 - This can become a limiting factor for very large H or T .

In practice, the convolution-based implementation is generally preferred for long sequences (large T) on GPUs, while the unrolled version might be better for very large hidden dimensions or when memory is a concern.